S3 SEED HARNESS

Each discovery run starts from the full SCoT seed harness below (iteration 0). The 151-line Python file prepares each function for analysis and validates the detector's JSON response. The optimizer modifies this file while preserving its overall structure.

```python
#!/usr/bin/env python3
"""
Standalone vulnerability-detection harness.

Plug this into any agent or model backend:
    from harness import build_prompt, parse_verdict
    prompt = build_prompt(code)          # send this to your model
    verdict = parse_verdict(raw_output)  # feed the model's raw text back in
"""
import json
import re
import sys
from typing import Any, Dict, Optional

PROMPT_TEMPLATE = 'You are an expert software engineer and security analyst.\n\nAnalyze the following code for security vulnerabilities.\n\nUse Structured Chain-of-Thought (SCoT): reason using explicit program structures:\n- Sequence (ordered steps)\n- Branch (if/else)\n- Loop (for/while)\n\nRules:\n- Base reasoning only on the given code.\n- Follow actual control flow (paths, checks, state changes).\n- Focus on security-relevant behavior (memory safety, bounds, lifetime, input validation, integer safety, authz/authn, unsafe APIs).\n- Do NOT rewrite or modify code.\n- After reasoning, output ONLY the JSON object described below.\n\nOutput format requirements:\n- First: write your SCoT reasoning in plain text with numbered steps.\n- Then: output ONLY one JSON object (no markdown, no extra text).\n\nRequired JSON schema:\n{\n  "verdict": {\n    "vulnerable": true/false,\n    "confidence": 0.0-1.0,\n    "cwe": ["CWE-..."],\n    "severity": "low|medium|high|critical|unknown",\n    "summary": "..."\n  },\n  "evidence": [\n    {\n      "file": null,\n      "function": null,\n      "location": {"start_line": null, "end_line": null},\n      "snippet": "...",\n      "reason": "..."\n    }\n  ],\n  "notes": {\n    "assumptions": [],\n    "limitations": []\n  }\n}\n\nCode:\n{{CODE}}\n'

REQUIRED_VERDICT_KEYS = {"vulnerable", "confidence", "cwe", "severity", "summary"}
REQUIRED_TOP_KEYS = {"verdict", "evidence", "notes"}


def build_prompt(code: str) -> str:
    """Render the harness prompt for a single code sample."""
    return PROMPT_TEMPLATE.replace("{{CODE}}", code)


def _strip_eos_tokens(raw: str) -> str:
    for tok in ("<|endoftext|>", "<|end|>", "<|im_end|>", "<|im_start|>"):
        raw = raw.replace(tok, "")
    return raw


def _sanitize_control_chars(raw: str) -> str:
    def _replace(m):
        c = m.group(0)
        return {'\t': '\\t', '\b': '\\b', '\f': '\\f'}.get(c, f'\\u{ord(c):04x}')
    return re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f]', _replace, raw)


def _find_verdict_object(text: str, strict: bool = True) -> Optional[Dict[str, Any]]:
    decoder = json.JSONDecoder(strict=strict)
    for m in re.finditer(r'\{', text):
        start = m.start()
        try:
            obj, _ = decoder.raw_decode(text, start)
        except json.JSONDecodeError:
            continue
        if not isinstance(obj, dict) or "verdict" not in obj:
            continue
        v = obj["verdict"]
        if isinstance(v, str) and "|" in v:
            continue
        if isinstance(v, dict) and isinstance(v.get("vulnerable"), bool):
            return obj
    return None


def _extract_verdict_direct(text: str) -> Optional[Dict[str, Any]]:
    for m in re.finditer(r'"verdict"\s*:\s*\{', text):
        vstart = m.end() - 1
```

```
        depth = 0
        for i in range(vstart, min(vstart + 5000, len(text))):
            if text[i] == '{':
                depth += 1
            elif text[i] == '}':
                depth -= 1
            if depth == 0:
                verdict_str = text[vstart:i + 1]
                for strict in (True, False):
                    try:
                        obj = json.loads(verdict_str, strict=strict)
                        if isinstance(obj, dict) and isinstance(obj.get("vulnerable"), bool):
                            return {"verdict": obj}
                    except json.JSONDecodeError:
                        continue
                break
    return None


def extract_json(raw: str) -> Optional[Dict[str, Any]]:
    """Find the first valid JSON verdict object in raw model output."""
    cleaned = _strip_eos_tokens(raw)

    result = _find_verdict_object(cleaned, strict=True)
    if result:
        return result

    result = _find_verdict_object(cleaned, strict=False)
    if result:
        return result

    sanitized = _sanitize_control_chars(cleaned)
    result = _find_verdict_object(sanitized, strict=True)
    if result:
        return result

    stripped = re.sub(r'```(?:json)?\s*', '', cleaned)
    result = _find_verdict_object(stripped, strict=False)
    if result:
        return result

    return _extract_verdict_direct(cleaned)


def validate_schema(obj: Dict[str, Any]) -> list:
    errors = []
    for k in REQUIRED_TOP_KEYS:
        if k not in obj:
            errors.append(f"missing top-level key: {k}")
    verdict = obj.get("verdict", {})
    if isinstance(verdict, dict):
        for k in REQUIRED_VERDICT_KEYS:
            if k not in verdict:
                errors.append(f"missing verdict key: {k}")
        if "vulnerable" in verdict and not isinstance(verdict["vulnerable"], bool):
            errors.append(f"verdict.vulnerable should be bool, got {type(verdict['vulnerable']).__name__}")
    else:
        errors.append(f"verdict should be dict, got {type(verdict).__name__}")
    return errors


def parse_verdict(raw_output: str) -> Dict[str, Any]:
    """Parse a model's raw text response into a structured verdict.

    Returns dict with: parsed, errors, vulnerable, confidence.
    """
    obj = extract_json(raw_output)
    if obj is None:
        return {
            "parsed": None,
            "errors": ["no valid JSON verdict found in output"],
            "vulnerable": None,
            "confidence": None,
        }
    schema_errors = validate_schema(obj)
    verdict = obj.get("verdict", {})
    return {
        "parsed": obj,
        "errors": schema_errors,
        "vulnerable": verdict.get("vulnerable") if isinstance(verdict, dict) else None,
```

```
        "confidence": verdict.get("confidence") if isinstance(verdict, dict) else None,
    }


def main() -> None:
    """CLI: render the prompt for a code file (or stdin) and print it."""
    code = open(sys.argv[1]).read() if len(sys.argv) > 1 else sys.stdin.read()
    print(build_prompt(code))


if __name__ == "__main__":
    main()
```